

Edge cases (200 doesn't mean safe to trust) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
import json

# problem 1: invalid JSON
try:
    json.loads("<html>error page</html>")
except json.JSONDecodeError as e:
    print("bad json:", e)

# problem 2: valid JSON, missing key
data = {"choices": []}

value = data.get("message")
print("using .get():", value)  # None, no crash

try:
    value = data["message"]
except KeyError as e:
    print("using []: crashed with", e)
```

Expected output:

```
bad json: Expecting value: line 1 column 1 (char 0)
using .get(): None
using []: crashed with 'message'
```

This covers both problems correctly. It's missing type hints and doesn't explain *when* to pick `.get()` vs `[...]` in a comment — both fine for a first pass exploring the behavior.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — two named functions, so the choice is explicit

```
import json
from typing import Any

def parse_body(raw_text: str) -> dict[str, Any]:
    """Parse a response body, raising a clear error if it isn't valid JSON."""
    try:
        return json.loads(raw_text)
    except json.JSONDecodeError as e:
        raise ValueError(f"response body is not valid JSON: {e}") from e

def get_required_field(data: dict[str, Any], key: str) -> Any:
    """Read a field the code genuinely can't work without – fail loudly if missing
    if key not in data:
        raise KeyError(f"expected field '{key}' was missing from the response")
    return data[key]

def get_optional_field(data: dict[str, Any], key: str, default: Any = None) -> Any:
    """Read a field that's genuinely optional – a missing value is normal, not an error
    return data.get(key, default)

if __name__ == "__main__":
    try:
        parse_body("<html>error page</html>")
    except ValueError as e:
        print(f"caught: {e}")

    payload = {"choices": []}
    print("optional field:", get_optional_field(payload, "message", default="no message"))

    try:
        get_required_field(payload, "message")
```

```
except KeyError as e:  
    print(f"caught: {e}")
```

Expected output:

```
caught: response body is not valid JSON: Expecting value: line 1 column 1 (char 0)  
optional field: no message  
caught: "expected field 'message' was missing from the response"
```

Difference from Basic: wrapping the two behaviors in named functions (`get_required_field` vs. `get_optional_field`) turns an implicit choice (`[key]` vs `.get(key)`), easy to pick inconsistently across a codebase) into an explicit, self-documenting decision every caller makes on purpose. `parse_body()` also re-raises `json.JSONDecodeError` as a plain `ValueError` with `from e` — this keeps the original traceback attached while giving callers one predictable error type to catch, instead of needing to know about `requests` ' or `json` 's specific exception classes.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a nested-path helper that checks type, not just presence

```
from typing import Any

def get_nested(data: Any, path: list[str | int], expected_type: type) -> Any:
    """Walk a nested dict/list path, raising one clear error naming exactly
    where it broke – either a missing step, or the final value being the
    wrong type."""
    current = data
    walked_so_far = ""

    for key in path:
        walked_so_far += f".{key}"
        if isinstance(key, str) and isinstance(current, dict) and key in current:
            current = current[key]
        elif isinstance(key, int) and isinstance(current, list) and key < len(current):
            current = current[key]
        else:
            raise ValueError(f"expected response shape missing at {walked_so_far}")

    if not isinstance(current, expected_type):
        raise ValueError(
            f"expected {expected_type.__name__} at {walked_so_far}, "
            f"got {type(current).__name__}"
        )

    return current

# happy path
payload = {"choices": [{"message": {"content": "hi there"}}]}
content = get_nested(payload, ["choices", 0, "message", "content"], str)
print(content)

# a level missing partway through
broken_payload = {"choices": []}
try:
    get_nested(broken_payload, ["choices", 0, "message", "content"], str)
```

```
except ValueError as e:
    print(f"caught: {e}")

# present, but the wrong type
wrong_type_payload = {"choices": "the server returned an error string here instead"}
try:
    get_nested(wrong_type_payload, ["choices"], list)
except ValueError as e:
    print(f"caught: {e}")
```

Expected output:

```
hi there
caught: expected response shape missing at .choices.0
caught: expected list at .choices, got str
```

Both failures name the exact spot in the path that broke — `.choices.0` for the missing list index, `.choices` (with the actual type found) for the wrong-type case — instead of a bare `TypeError: string indices must be integers` popping up wherever the bad value next gets used.

Approach 2 — `pydantic`, the shape declared instead of hand-walked

```
from pydantic import BaseModel, ValidationError

class Message(BaseModel):
    content: str

class Choice(BaseModel):
    message: Message

class ChatResponse(BaseModel):
    choices: list[Choice]

payload = {"choices": [{"message": {"content": "hi there"}}]}
response = ChatResponse.model_validate(payload)
print(response.choices[0].message.content)

broken_payload = {"choices": "the server returned an error string here instead"}
try:
    ChatResponse.model_validate(broken_payload)
```

```
except ValidationError as e:
    print(f"caught: {e.error_count()} validation error(s)")
```

Expected output:

```
hi there
caught: 1 validation error(s)
```

Instead of writing `get_nested()` yourself, you describe the response's whole expected shape as classes once, and `model_validate()` checks every level — presence and type — in one call, raising `ValidationError` with every problem it found, not just the first one.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `get_required_field` / `get_optional_field` handle exactly one key, one level deep — correct, but they don't help once the field you need is nested, or once a field is present but the wrong type. Approach 1 generalizes to an arbitrary path and checks type as well as presence, still with no new dependency. Approach 2 replaces the hand-written walk entirely — you declare the shape once as a set of classes, and validation (including reporting every problem at once, not just the first) is handled by a well-tested library, at the cost of learning `pydantic`'s API and adding a dependency.

Which one should you actually write? For a response with 1-2 fields you care about, Intermediate's two named functions are enough. For `http_client.py` in this document's Build Task, or any response with a genuinely nested shape (which most real LLM API responses have), Approach 1's `get_nested()` pattern is worth having even without a new dependency. Reach for Approach 2 (`pydantic`) once you're validating a response shape with several fields and would rather declare it once as a typed model than hand-write a `get_nested()` call for each field — Doc04's `chat_client.py` is a natural place to make that switch.